

- ▶ Operadores, condiciones y bucles
- ▶ Elementos de pseudocódigo y diagramas de flujo

Bibliografía

- *Introducción a la programación con Python. Sec. 1.4*, Andrés Marzal, Isabel Gracia y Pedro García.
- *Fundamentos de programación. Cap. 2*, Luis Joyanes.
- *Métodos numéricos para ingenieros. Sec. 2.2*, Steven C. Chapra and Raymond P. Canale.

Operadores, Condicionales y Bucles

Desde los cursos básicos en Matemáticas aprendemos las operaciones básicas: sumar, restar, multiplicar y dividir. Estos son **Operadores aritméticos**, los cuales representamos mediante símbolos. Pero estos no son los únicos operadores que existen. En la programación, además de los operadores aritméticos, están los operadores de comparación y los operadores lógicos.

a) Operadores aritméticos: Se utilizan para realizar cálculos matemáticos básicos.

- Suma (+), Resta (-), Multiplicación (*), División (/).
- Potenciación (^ o **).
- Módulo (% o mod): Devuelve el residuo de una división entera (por ejemplo, $7 \bmod 2 = 1$).

b) Operadores de comparación (relacionales): Permiten comparar dos valores. El resultado de esta comparación siempre es un valor de verdad o booleano: *Verdadero* (True) o *Falso* (False).

- Igual a (==), Diferente de (!=).
- Mayor que (>), Menor que (<).
- Mayor o igual que (>=), Menor o igual que (<=).

c) Operadores lógicos: Permiten evaluar o combinar múltiples expresiones booleanas.

- **AND (Y / &&):** Devuelve *Verdadero* únicamente si **ambas** condiciones son verdaderas.
- **OR (O / ||):** Devuelve *Verdadero* si **al menos una** de las condiciones es verdadera.
- **NOT (NO / !):** Invierte el valor de verdad (convierte *Verdadero* en *Falso* y viceversa).

Normalmente las operaciones lógicas se utilizan para tomar decisiones en un programa, por ejemplo,

Ejemplos cotidianos:

- **Ejemplo AND (Y):**

«Colombia» Y «Brasil se encuentran en suramérica» $\longrightarrow$ **True**

Ambas proposiciones son verdaderas. Sin embargo:

«Colombia se encuentra en norteamérica» Y «Mexico se encuentra en norteamérica» $\longrightarrow$ **False**

Falla una de las condiciones, por lo que todo el argumento se vuelve falso.

- **Ejemplo OR (O):**

«El Sol es una estrella» O «La Luna es un planeta» $\longrightarrow$ **True**

Basta con que una de las dos partes sea verdadera para que toda la expresión sea verdadera.

Una vez que sabemos cómo construir evaluaciones lógicas con los operadores, podemos controlar el flujo con el que la computadora ejecuta las instrucciones:

- **Condicionales (Estructuras de selección):** Le permiten al programa decidir si ejecuta un bloque de código u otro según el resultado de una comparación. Por ejemplo, verificar si el discriminante de una ecuación es negativo ($D < 0$) para decidir si calcular raíces reales o notificar que son complejas.
- **Bucles (Estructuras de repetición):** Permiten iterar (repetir) un conjunto de instrucciones de forma automática mientras se cumpla una condición o durante un número fijo de veces. Por ejemplo, evaluar una función de onda en 1000 puntos espaciales distintos.

Elementos de pseudocódigo y diagramas de flujo

En la clase anterior estudiamos cómo las computadoras representan la información en memoria, desde el sistema binario para enteros hasta el estándar IEEE 754 para números flotantes. Sin embargo, para solucionar un problema físico o matemático no basta con almacenar datos: es necesario procesarlos mediante una secuencia ordenada y finita de instrucciones, es decir, un **algoritmo**. Un algoritmo es un conjunto de pasos orientado a resolver un problema específico.

A la hora de programar es fundamental mantener una estructura clara. En el ámbito de la física computacional y el cálculo numérico, cualquier algoritmo puede construirse combinando únicamente tres estructuras lógicas fundamentales:

1. **Secuencia:** Ejecución de instrucciones paso a paso, una tras otra.
2. **Selección (Condicionales):** Toma de decisiones que alteran el flujo del programa según se cumpla o no una condición (por ejemplo, verificar si el discriminante de una ecuación es negativo o si una velocidad supera la velocidad de la luz).

3. Repetición (Bucles):

Iteración de un conjunto de pasos bajo ciertas condiciones (por ejemplo, calcular una propiedad física para distintos valores de una variable).

Para diseñar y comunicar estos algoritmos antes de traducirlos a un lenguaje de programación específico (como Python, C++ o Julia), empleamos dos herramientas principales: los **diagramas de flujo**, que ofrecen una representación gráfica e intuitiva del proceso, y el **pseudocódigo**, que utiliza una sintaxis cercana al lenguaje humano estructurado.

Definición: Diagrama de flujo

Un **diagrama de flujo** es la representación visual o gráfica de un algoritmo. Utiliza símbolos estandarizados para describir los pasos del proceso:

- **Óvalo / Óvalo alargado:** Inicio o fin del programa. 
- **Líneas de flujo (flechas):** Indican la secuencia de ejecución. 
- **Rectángulo:** Procesos, cálculos y/o manipulación de datos. 
- **Paralelogramo:** Entrada/Salida de información. 
- **Rombo:** Decisión o comparación. 

Definición: Pseudocódigo

En lugar de utilizar símbolos gráficos, el **pseudocódigo** aboga por el uso de palabras y expresiones semejantes a un lenguaje de programación estructurado.

Ejemplo 1: Algoritmo cotidiano para hacer buñuelos. Supongamos que queremos preparar buñuelos y deseamos expresar el procedimiento de manera clara y estructurada para que otra persona lo pueda hacer. Un posible pseudocódigo sería el siguiente:

1. Inicio
2. Comprar materiales
3. Preparar masa
4. Hacer bolitas con la masa
5. Calentar aceite
6. Introducir las bolitas en el aceite
7. Freír
8. Repetir desde el paso 3
9. Sacar los buñuelos
10. Comer buñuelos

11. Fin

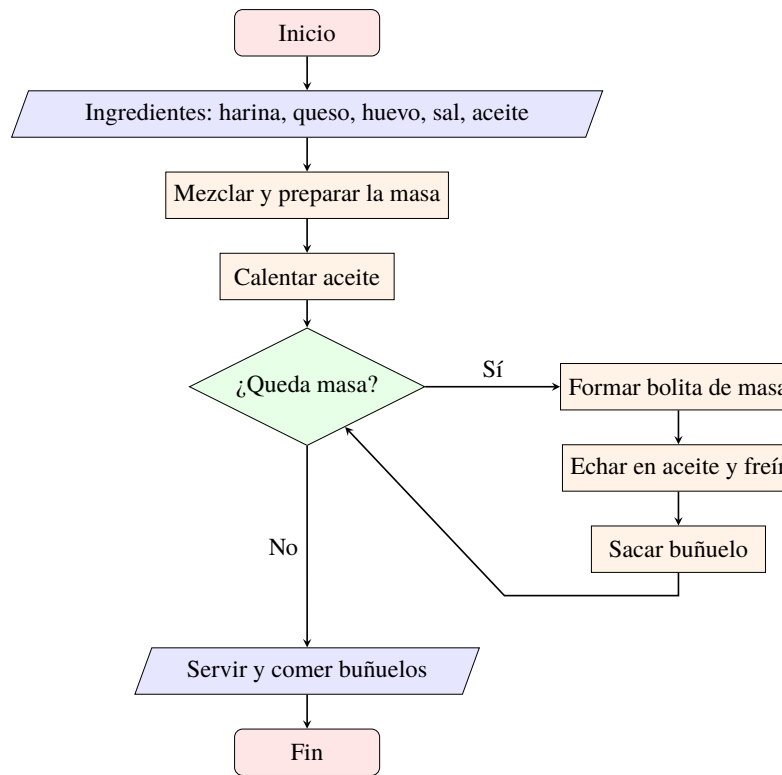
Note que la construcción del algoritmo no es única. Depende de qué tan específico se quiera ser. Note por ejemplo que el paso 8) indica repetir desde el paso 4), lo cual implica que se deben hacer más bolitas. Sin embargo, algo no cuadra. ¿Qué pasa si se acaba la masa? ¿Cómo sabe la persona que está haciendo los buñuelos que ya no hay más masa y que debe parar de freír? Para resolverlo, introducimos una estructura repetitiva: **Mientras (while)**. Mientras exista masas, repetir desde el paso 4).

1. Inicio
2. Comprar materiales
3. Preparar masa
4. Calentar aceite
5. Mientras quede masa hacer:
 - a) Hacer bolitas con la masa
 - b) Introducir las bolitas en el aceite
 - c) Freír
 - d) Sacar los buñuelos
6. Comer buñuelos
7. Fin

La instrucción **Mientras** permite ejecutar un bloque de pasos de forma cíclica apoyándose en una evaluación lógica:

- **Evaluación previa:** Antes de entrar al bloque indentado (pasos a-d), la computadora (o lka persona que cocina) evalúa la condición: *¿Queda masa?*
- **Rama Verdadera (True):** Si la respuesta es **Sí**, se ejecutan en orden las instrucciones del bloque interno (hacer bolita, freír, sacar). Al terminar el paso d), el flujo **vuelve automáticamente a subir al paso 5** para preguntar de nuevo: *¿Aún queda masa?*
- **Rama Falsa (False):** En el momento en que la respuesta a la condición sea **No** (se agotó la masa), el programa rompe el bucle, omite los pasos a-d y salta directamente a la siguiente instrucción fuera del bloque (paso 6: *Comer buñuelos*).

Esta es la esencia del algoritmo estructurado: el bucle evalúa continuamente la condición y solo se detiene cuando esta deja de cumplirse. Hay que tener cuidado con la evaluación y asegurarse de que en algún momento faltará masa, pues la persona podría quedarse infinitamente haciendo buñuelos. Si quieres, puedes agregar más detalles, como recuerda que los buñuelos se voltean solos, o que se deben sacar cuando estén dorados. Finalmente, también podemos usar la representación gráfica mediante un diagrama de flujo.

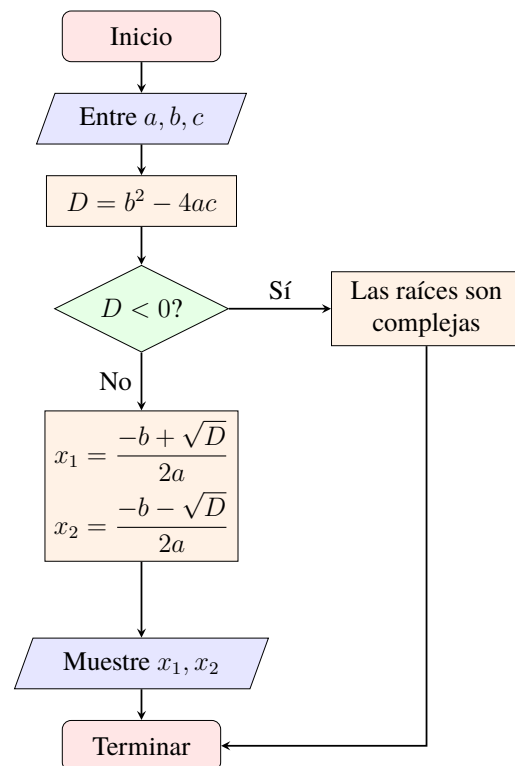


Ejemplo 2: Algoritmo para encontrar raíces de una ecuación cuadrática $ax^2 + bx + c = 0$. Recordemos que las soluciones son $x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$.

Pseudocódigo:

Diagrama de flujo:

- 1) Inicio
- 2) Entre a, b y c
- 3) Calcular $D = b^2 - 4ac$
- 4) Si $D < 0$ entonces
 - a) Muestre: "Las raíces son complejas"
- 5) Sino
 - a) Haga $x_1 = \frac{-b + \sqrt{D}}{2a}$
 - b) Haga $x_2 = \frac{-b - \sqrt{D}}{2a}$
 - c) Muestre x_1, x_2
- 6) Fin Si
- 7) Terminar



Note que en el pseudocódigo se emplea la estructura de selección **If: Si, Else: Sino** para decidir si las raíces son complejas o reales, mientras que en el diagrama de flujo se representa mediante un rombo (decisión) con dos ramas: una para el caso de raíces complejas y otra para el caso de raíces reales.

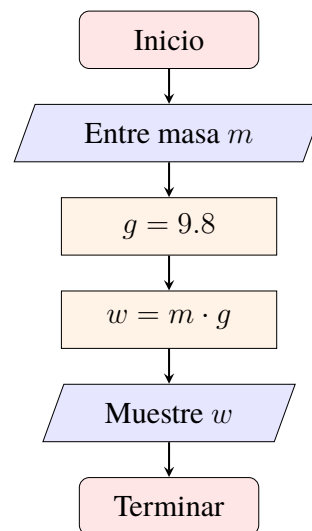
La estructura condicional **If: Si, Else:Sino** permite modificar el flujo de ejecución de un programa según el resultado de una evaluación lógica (*Verdadero* o *Falso*).

- **Evaluación de la condición (Paso 4):** El programa evalúa la expresión relacional $D < 0$. El resultado solo puede ser *Verdadero* (True) o *Falso* (False).
- **Rama Verdadero (Si $D < 0$ es True):** Se ejecuta únicamente el bloque interno indentado bajo el **Si** (paso 4a: mostrar que las raíces son complejas). Una vez ejecutadas estas instrucciones, el programa salta automáticamente al final de la estructura (**Fin Si**) y continúa en el paso 7 (*Terminar*), ignorando por completo la sección del **Sino**.
- **Rama Falso (Si $D < 0$ es False):** El programa omite el paso 4a y pasa directo a ejecutar las instrucciones dentro de la sección **Sino** (pasos 5a, 5b y 5c para calcular e imprimir x_1 y x_2). Al terminar, continúa con el flujo normal.

Es importante añadir que las ramas **Si** y **Sino** son mutuamente excluyentes. El programa ejecutará un camino o el otro, pero jamás ambos en una misma ejecución.

Ejemplo 3. Calcular el peso de un cuerpo dada su masa

- 1) Inicio
- 2) Entre la masa m (kg)
- 3) Asignar $g = 9.8 \text{ m/s}^2$
- 4) Calcular $w = m \cdot g$
- 5) Muestre “El peso en N es:”, w
- 6) Terminar



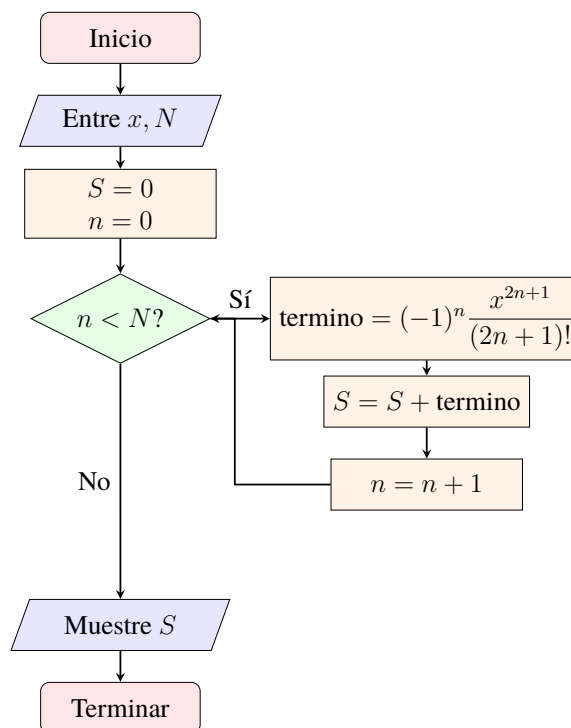
Ejemplo 4. La función $\sin(x)$: Una computadora solo sabe sumar, restar, multiplicar o dividir. Por esta razón, el cálculo de muchas funciones complicadas en física y matemática (como $\sin x$, $\cos x$ o e^x) se realiza mediante el uso de series de potencias (series de Taylor). Para la función seno, la expansión alrededor de $x = 0$ está dada por la suma infinita:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+1}}{(2n+1)!}$$

donde el ángulo x está expresado en radianes. Dado que un sistema computacional no puede sumar infinitos términos, en la práctica la serie se trunca sumando únicamente los primeros N términos, asumiendo que el residuo truncado es cuantitativamente despreciable.

Pseudocódigo:

- 1) Inicio
- 2) Entre el ángulo, x , en radianes
- 3) Entre el número de términos, N
- 4) $S = 0$
- 5) $n = 0$
- 6) Mientras $n < N$ haga:
 - a) $\text{termino} = (-1)^n \cdot \frac{x^{2n+1}}{(2n+1)!}$
 - b) $S = S + \text{termino}$
 - c) $n = n + 1$
- 7) Muestre $\sin(x) \approx S$
- 8) Terminar

Diagrama de flujo:

Note que definimos un acumulador S que va guardando los términos de la suma. Por ejemplo, si lo iniciamos en cero, se tiene $S = 0$, luego se suma el primer término $S = 0 + \text{termino1} = 0 + x$, luego el segundo $S = 0 + \text{termino1} + \text{termino2} = 0 + x - \frac{x^3}{3!}$, y así sucesivamente hasta que se sumen los N términos. Acá es importante mencionar que la expresión $S = S + a$ en programación se interpreta como: "actualiza el valor de S sumándole a ", no representa una igualdad matemática. La igualdad matemática se tiene reservada el símbolo $==$. Además, observen la diferencia entre contar y acumular. la instrucción $n = n + 1$ indica que se actualiza el valor de n sumándole siempre 1.

Problemas:

1. Construya un algoritmo para hacer empanadas.
2. Modifique el algoritmo del problema 3 y diseñe otro diagrama de flujo, pero que ahora distinga si la masa ingresada es positiva, negativa o cero. Si $m \leq 0$, el programa debe mostrar un mensaje de error indicando que la masa no es válida; de lo contrario, debe realizar el cálculo del peso y mostrar el resultado.
3. Elabore un pseudocódigo y un diagrama de flujo para calcular el valor de la suma $1 + 2 + 3 \dots 100$.
4. Elabore un pseudocódigo y un diagrama de flujo para calcular el área de un triángulo rectángulo dada su base y altura.
5. Elabore un pseudocódigo y un diagrama de flujo para calcular el tiempo que le toma a un cuerpo alcanzar una altura h , siendo lanzado desde una altura h_0 con velocidad inicial v_0 . Recuerde que desde la mecánica básica, sabemos que

$$h = h_0 + v_0 t - \frac{1}{2} g t^2 \implies \frac{1}{2} g t^2 - v_0 t + (h - h_0) = 0 \implies t_{1,2} = \frac{v_0 \pm \sqrt{v_0^2 - 2g(h - h_0)}}{g}$$

Indicación: El algoritmo debe evaluar si la cantidad bajo la raíz es negativa ($v_0^2 < 2g(h - h_0)$). Si es así, debe mostrar un mensaje indicando que el objeto jamás alcanza la altura h . En caso contrario, debe calcular y mostrar el tiempo (o tiempos) físicamente válidos.

6. Un objeto se deja caer desde una altura y_0 con velocidad inicial cero. Diseñe el pseudocódigo y el diagrama de flujo que imprima segundo a segundo ($t = 0, 1, 2, \dots$) la altura del cuerpo $y(t) = y_0 - \frac{1}{2} g t^2$ mientras el objeto no haya tocado el suelo ($y \geq 0$).
7. Modifique el algoritmo del ejemplo 4 para que muestre un error si el número de términos, N , no es un número entero positivo.
8. Elabore un pseudocódigo y diagrama de flujo para aproximar el valor de la suma de N términos discretos:

$$S = \sum_{i=1}^N \frac{1}{i^2} = 1 + \frac{1}{4} + \frac{1}{9} + \dots + \frac{1}{N^2}$$

El algoritmo debe solicitar el número entero N , inicializar un acumulador en cero y sumar los términos iterativamente utilizando un bucle **while**.